

Scalability & Performance Optimization

Comprehensive Guide to Scaling Data Ingestion Systems

Table of Contents

- 1. [Scaling Strategies](#)
- 2. [Partitioning & Sharding](#)
- 3. [Backpressure Handling](#)
- 4. [Throughput Optimization](#)
- 5. [Latency Reduction](#)
- 6. [Capacity Planning](#)
- 7. [Performance Benchmarks](#)
- 8. [Resource Optimization](#)

1. Scaling Strategies {#scaling-strategies}

Horizontal vs Vertical Scaling

Vertical Scaling (Scale Up)

Single Large Machine

- 64 vCPU
- 256 GB RAM
- 10 TB SSD

Pros: Simple

Cons: Limited, Expensive

Horizontal Scaling (Scale Out)

8 vCPU

32GB

1TB

8 vCPU

32GB

1TB

8 vCPU

32GB

1TB

8 vCPU

32GB

1TB

Pros: Unlimited, Resilient

Cons: Coordination complexity

Kafka Scaling

Adding Brokers:

```
# Current: 3 brokers handling 10K msg/sec
# Target: 30K msg/sec → Need 9 brokers

# Step 1: Add new brokers
for i in {4..9}; do
  docker-compose up -d kafka-$i
done

# Step 2: Reassign partitions to new brokers
kafka-reassign-partitions.sh \
  --bootstrap-server localhost:9092 \
  --reassignment-json-file reassignment.json \
  --execute

# Step 3: Verify reassignment
kafka-reassign-partitions.sh \
  --bootstrap-server localhost:9092 \
  --reassignment-json-file reassignment.json \
  --verify
```

Partition Scaling:

```
# Increase partitions for existing topic
kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --alter \
  --topic user-events \
  --partitions 30

# Note: Can only increase, not decrease!
```

Production Kafka Scaling Configuration:

```
# broker.properties (per broker)

# Network threads (1 per CPU core)
num.network.threads=16

# I/O threads (2x number of disks)
num.io.threads=32

# Socket buffers
socket.send.buffer.bytes=1048576
socket.receive.buffer.bytes=1048576
socket.request.max.bytes=104857600

# Log settings
num.partitions=30
log.retention.hours=168
```

```
log.segment.bytes=1073741824

# Replication
default.replication.factor=3
min.insync.replicas=2
replica.lag.time.max.ms=30000
```

Flink Scaling

Horizontal Scaling:

```
# flink-conf.yaml

# Increase parallelism
parallelism.default: 32

# More TaskManagers
taskmanager.numberOfTaskSlots: 8

# Memory per TaskManager
taskmanager.memory.process.size: 16g
taskmanager.memory.flink.size: 12g

# Network buffers
taskmanager.network.memory.fraction: 0.2
taskmanager.network.memory.min: 256mb
taskmanager.network.memory.max: 2gb
```

Auto-scaling Flink on Kubernetes:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flink-taskmanager-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: flink-taskmanager
  minReplicas: 5
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Pods
```

```

    pods:
      metric:
        name: flink_taskmanager_job_task_backPressuredTimeMsPerSecond
      target:
        type: AverageValue
        averageValue: "5000"
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
    policies:
      - type: Percent
        value: 10
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 60
    policies:
      - type: Percent
        value: 50
        periodSeconds: 60
      - type: Pods
        value: 5
        periodSeconds: 60
    selectPolicy: Max

```

Spark Scaling

Dynamic Allocation:

```

spark = SparkSession.builder \
  .appName("Scalable Pipeline") \
  .config("spark.dynamicAllocation.enabled", "true") \
  .config("spark.dynamicAllocation.minExecutors", "5") \
  .config("spark.dynamicAllocation.maxExecutors", "100") \
  .config("spark.dynamicAllocation.initialExecutors", "10") \
  .config("spark.dynamicAllocation.executorIdleTimeout", "60s") \
  .config("spark.dynamicAllocation.cachedExecutorIdleTimeout", "300s") \
  .config("spark.dynamicAllocation.schedulerBacklogTimeout", "1s") \
  .config("spark.shuffle.service.enabled", "true") \
  .getOrCreate()

```

Resource Tuning:

```

spark-submit \
  --master yarn \
  --deploy-mode cluster \
  --executor-memory 8G \
  --executor-cores 4 \
  --num-executors 50 \
  --driver-memory 4G \

```

```
--conf spark.default.parallelism=400 \
--conf spark.sql.shuffle.partitions=400 \
--conf spark.memory.fraction=0.8 \
--conf spark.memory.storageFraction=0.3 \
--conf spark.executor.memoryOverhead=2G \
pipeline.py
```

2. Partitioning & Sharding {#partitioning}

Kafka Partitioning Strategies

1. Key-based Partitioning (Default):

```
// Records with same key always go to same partition
producer.send(new ProducerRecord<>(
    "user-events",
    userId, // Key - determines partition
    event   // Value
));

// Partition calculation:
// partition = hash(key) % num_partitions
```

2. Custom Partitioner:

```
public class CustomPartitioner implements Partitioner {

    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes, Cluster cluster) {

        // Get number of partitions
        int numPartitions = cluster.partitionCountForTopic(topic);

        // Custom logic: Geographic partitioning
        String userId = (String) key;
        String region = extractRegion(userId);

        switch (region) {
            case "US":
                return (userId.hashCode() % (numPartitions / 2));
            case "EU":
                return (numPartitions / 2) + (userId.hashCode() % (numPartitions /
2));
            default:
                return userId.hashCode() % numPartitions;
        }
    }
}
```

```

    private String extractRegion(String userId) {
        // Extract region from user ID
        return userId.substring(0, 2);
    }
}

// Use custom partitioner
Properties props = new Properties();
props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG,
    CustomPartitioner.class.getName());

```

3. Round-Robin Partitioning:

```

// No key = round-robin across partitions
producer.send(new ProducerRecord<>(
    "user-events",
    null,    // No key
    event
));

```

Partition Count Guidelines

Calculation Formula:

```

Partitions = max(
    Target_Throughput / Partition_Throughput,
    Total_Consumer_Parallelism
)

```

Example:

- Target: 100 MB/s
- Partition throughput: 10 MB/s
- Consumer parallelism: 20

Partitions = $\max(100/10, 20) = \max(10, 20) = 20$ partitions

Best Practices:

- ☒ Do:
 - Start with: $\text{num_partitions} = \text{num_consumers} * 2$
 - Use key-based partitioning for ordering
 - Plan for growth (partitions can only increase)
 - Monitor partition skew

- ☒ Don't:
 - Too few partitions (<10) = underutilization

- Too many partitions (>1000/broker) = overhead
- Frequent partition changes = rebalancing
- Unbalanced keys = hot partitions

Database Sharding

Horizontal Sharding Strategy:

```
-- Shard by user_id (hash-based)
CREATE TABLE orders_shard_0 (
    order_id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    amount DECIMAL(10,2),
    created_at TIMESTAMP,
    CHECK (MOD(user_id, 4) = 0)
);

CREATE TABLE orders_shard_1 (
    order_id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    amount DECIMAL(10,2),
    created_at TIMESTAMP,
    CHECK (MOD(user_id, 4) = 1)
);

-- Continue for shards 2, 3...
```

Application-level Sharding:

```
class ShardedDatabase:
    """
    Application-level database sharding
    """

    def __init__(self, shard_configs: List[Dict]):
        """
        Initialize connections to all shards

        Args:
            shard_configs: List of shard connection configs
        """
        self.shards = []
        for config in shard_configs:
            conn = psycopg2.connect(**config)
            self.shards.append(conn)

        self.num_shards = len(self.shards)

    def get_shard(self, user_id: int):
```

```

        """Get shard for user_id"""
        shard_id = user_id % self.num_shards
        return self.shards[shard_id]

def insert_order(self, order: Dict):
    """Insert order into correct shard"""
    user_id = order['user_id']
    shard = self.get_shard(user_id)

    cursor = shard.cursor()
    cursor.execute("""
        INSERT INTO orders (order_id, user_id, amount, created_at)
        VALUES (%(order_id)s, %(user_id)s, %(amount)s, %(created_at)s)
    """, order)
    shard.commit()

def query_user_orders(self, user_id: int):
    """Query orders for user (single shard)"""
    shard = self.get_shard(user_id)
    cursor = shard.cursor()

    cursor.execute("""
        SELECT * FROM orders
        WHERE user_id = %s
        ORDER BY created_at DESC
    """, (user_id,))

    return cursor.fetchall()

def query_all_orders(self, filters: Dict):
    """Query across all shards (scatter-gather)"""
    from concurrent.futures import ThreadPoolExecutor, as_completed

    def query_shard(shard):
        cursor = shard.cursor()
        cursor.execute("""
            SELECT * FROM orders
            WHERE created_at >= %s
            ORDER BY created_at DESC
        """, (filters['start_date'],))
        return cursor.fetchall()

    # Parallel query across all shards
    results = []
    with ThreadPoolExecutor(max_workers=self.num_shards) as executor:
        futures = [executor.submit(query_shard, shard)
                    for shard in self.shards]

        for future in as_completed(futures):
            results.extend(future.result())

    # Merge and sort results
    results.sort(key=lambda x: x['created_at'], reverse=True)
    return results

```



```
# Usage
shards = ShardedDatabase([
    {'host': 'db-shard-0', 'port': 5432, 'database': 'orders', 'user': 'app'},
    {'host': 'db-shard-1', 'port': 5432, 'database': 'orders', 'user': 'app'},
    {'host': 'db-shard-2', 'port': 5432, 'database': 'orders', 'user': 'app'},
    {'host': 'db-shard-3', 'port': 5432, 'database': 'orders', 'user': 'app'},
])

shards.insert_order({
    'order_id': 12345,
    'user_id': 789,
    'amount': 99.99,
    'created_at': datetime.now()
})
```

3. Backpressure Handling {#backpressure}

Understanding Backpressure

```
Producer → [Fast] → Kafka → [Slow] → Consumer → [Very Slow] → Database
                        ↓
                    Queue fills up
                        ↓
                    Backpressure!
```

Flink Backpressure

Detection:

```
# Check backpressure via Flink UI
# http://flink-jobmanager:8081/#/jobs/<job-id>/backpressure

# Or via metrics
curl http://flink-jobmanager:8081/jobs/<job-id>/vertices/<vertex-id>/backpressure
```

Handling Strategies:

1. Increase Parallelism:

```
DataStream<Event> events = env.fromSource(source, ...);

// Increase parallelism for slow operators
DataStream<Result> results = events
    .map(new SlowMapper())
    .setParallelism(32) // More parallel instances
```

```
.keyBy(...)
.process(new Aggregator())
.setParallelism(16);
```

2. Async I/O (for external calls):

```
import org.apache.flink.streaming.api.functions.async.AsyncFunction;

public class AsyncDatabaseLookup extends RichAsyncFunction<Event, EnrichedEvent> {

    private transient DatabaseAsyncClient client;

    @Override
    public void open(Configuration parameters) {
        client = new DatabaseAsyncClient();
    }

    @Override
    public void asyncInvoke(Event event, ResultFuture<EnrichedEvent> resultFuture)
    {
        // Async database lookup
        CompletableFuture<UserInfo> future =
        client.getUserInfo(event.getUserId());

        future.thenAccept(userInfo -> {
            resultFuture.complete(Collections.singleton(
                new EnrichedEvent(event, userInfo)
            ));
        });
    }
}

// Use async operator
DataStream<EnrichedEvent> enriched = events
    .keyBy(Event::getUserId)
    .flatMap(AsyncDataStream.unorderedWait(
        new AsyncDatabaseLookup(),
        5000, // Timeout: 5 seconds
        TimeUnit.MILLISECONDS,
        100 // Capacity: 100 concurrent requests
    ));
```

3. Network Buffer Tuning:

```
# flink-conf.yaml

# Increase network buffers
taskmanager.network.memory.fraction: 0.2
taskmanager.network.numberOfBuffers: 8192
```

```
# Buffer timeout (trade latency for throughput)
execution.buffer-timeout: 100ms
```

Kafka Consumer Backpressure

1. Pause/Resume Pattern:

```
public class BackpressureAwareConsumer {

    private final KafkaConsumer<String, String> consumer;
    private final BlockingQueue<ConsumerRecord<String, String>> buffer;
    private final int maxBufferSize = 10000;

    public void consume() {
        while (running) {
            // Check buffer size
            if (buffer.size() > maxBufferSize * 0.9) {
                // Pause consumption when buffer is 90% full
                consumer.pause(consumer.assignment());
                logger.warn("Paused consumption due to backpressure");
            } else if (buffer.size() < maxBufferSize * 0.5) {
                // Resume when buffer drops to 50%
                consumer.resume(consumer.assignment());
                logger.info("Resumed consumption");
            }

            // Poll
            ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));

            // Add to buffer
            for (ConsumerRecord<String, String> record : records) {
                try {
                    buffer.put(record);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        }
    }
}
```

2. Rate Limiting:

```
from time import sleep, time
from collections import deque
```

```

class RateLimitedConsumer:
    """
    Consumer with rate limiting
    """

    def __init__(self, consumer, max_rate_per_second: int):
        self.consumer = consumer
        self.max_rate = max_rate_per_second
        self.timestamps = deque(maxlen=max_rate_per_second)

    def consume(self):
        """Consume with rate limiting"""
        while True:
            # Rate limit check
            current_time = time()

            if len(self.timestamps) >= self.max_rate:
                # Check if we need to wait
                oldest_timestamp = self.timestamps[0]
                time_diff = current_time - oldest_timestamp

                if time_diff < 1.0:
                    # Wait until rate limit window passes
                    sleep_time = 1.0 - time_diff
                    sleep(sleep_time)

            # Poll message
            msg = self.consumer.poll(1.0)

            if msg is None:
                continue

            # Process message
            self.process_message(msg)

            # Track timestamp
            self.timestamps.append(time())

    def process_message(self, msg):
        """Process message"""
        # Your processing logic
        pass

# Usage
consumer = Consumer({'bootstrap.servers': 'localhost:9092', ...})
rate_limited = RateLimitedConsumer(consumer, max_rate_per_second=1000)
rate_limited.consume()

```

4. Throughput Optimization {#throughput}

Producer Throughput

Batching Configuration:

```

Properties props = new Properties();

// Batching settings
props.put(ProducerConfig.BATCH_SIZE_CONFIG, "32768"); // 32KB
props.put(ProducerConfig.LINGER_MS_CONFIG, "10");      // Wait 10ms

// Compression (huge throughput boost)
props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "lz4");

// Buffer memory
props.put(ProducerConfig.BUFFER_MEMORY_CONFIG, "67108864"); // 64MB

// Multiple in-flight requests
props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "5");

KafkaProducer<String, String> producer = new KafkaProducer<>(props);

```

Throughput Comparison:

Configuration	Throughput
No batching, no compression	10K msg/s
Batching (10ms linger)	50K msg/s
Batching + LZ4 compression	200K msg/s
Batching + Snappy compression	180K msg/s
Batching + GZIP compression	120K msg/s (higher CPU)

Consumer Throughput**Parallel Processing:**

```

from concurrent.futures import ThreadPoolExecutor
from confluent_kafka import Consumer
import queue

class ParallelConsumer:
    """
    Multi-threaded consumer for higher throughput
    """

    def __init__(self, config, num_workers: int = 10):
        self.consumer = Consumer(config)
        self.num_workers = num_workers
        self.message_queue = queue.Queue(maxsize=1000)
        self.executor = ThreadPoolExecutor(max_workers=num_workers)

```

```

def consume(self, topics: list):
    """Start consuming"""
    self.consumer.subscribe(topics)

    # Start worker threads
    for _ in range(self.num_workers):
        self.executor.submit(self.worker)

    # Main loop: poll and queue messages
    try:
        while True:
            msg = self.consumer.poll(0.1)

            if msg is None:
                continue
            if msg.error():
                continue

            # Add to queue for processing
            self.message_queue.put(msg)

        except KeyboardInterrupt:
            pass
    finally:
        self.shutdown()

def worker(self):
    """Worker thread that processes messages"""
    while True:
        try:
            msg = self.message_queue.get(timeout=1)
            self.process_message(msg)
            self.message_queue.task_done()
        except queue.Empty:
            continue

def process_message(self, msg):
    """Process a single message"""
    # Your processing logic
    value = msg.value().decode('utf-8')
    # ... process ...

def shutdown(self):
    """Shutdown consumer"""
    self.executor.shutdown(wait=True)
    self.consumer.close()

# Usage
config = {
    'bootstrap.servers': 'localhost:9092',
    'group.id': 'parallel-consumer',
    'max.poll.records': 500, # Fetch more records per poll
    'fetch.min.bytes': 1024, # Wait for more data
    'fetch.max.wait.ms': 500 # Or timeout

```

```

}

consumer = ParallelConsumer(config, num_workers=20)
consumer.consume(['user-events'])

```

Flink Throughput

Tuning Parameters:

```

# flink-conf.yaml

# Parallelism
parallelism.default: 64

# Network
taskmanager.network.memory.fraction: 0.2
taskmanager.network.numberOfBuffers: 16384

# Checkpointing (less frequent = higher throughput)
execution.checkpointing.interval: 300000 # 5 minutes
execution.checkpointing.mode: AT_LEAST_ONCE # vs EXACTLY_ONCE

# State backend (RocksDB for large state)
state.backend: rocksdb
state.backend.rocksdb.predefined-options: SPINNING_DISK_OPTIMIZED_HIGH_MEM

# Async checkpoints
state.backend.async: true
state.backend.incremental: true

```

Code-level Optimization:

```

StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

// Object reuse (reduce GC pressure)
env.getConfig().enableObjectReuse();

// Disable operator chaining (for debugging)
// env.disableOperatorChaining();

// Buffer timeout (latency vs throughput trade-off)
env.setBufferTimeout(100); // 100ms

DataStream<Event> events = env.fromSource(...);

// Use map instead of flatMap when possible (less overhead)
DataStream<Result> results = events
    .map(new FastMapper())

```

```
.keyBy(...)  
.window(...)  
.reduce(new FastReducer()); // reduce() is faster than aggregate()
```

5. Latency Reduction {#latency}

End-to-End Latency Breakdown

Total Latency = Producer + Network + Broker + Network + Consumer + Processing

Example:

- Producer batching: 10ms
- Network (producer→broker): 5ms
- Broker write: 2ms
- Network (broker→consumer): 5ms
- Consumer poll interval: 100ms
- Processing: 50ms

Total: ~172ms

Low-Latency Kafka Configuration

Producer:

```
Properties props = new Properties();  
  
// Disable batching for lowest latency  
props.put(ProducerConfig.LINGER_MS_CONFIG, "0");  
props.put(ProducerConfig.BATCH_SIZE_CONFIG, "0");  
  
// Single in-flight request  
props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, "1");  
  
// Faster acks (trade durability for latency)  
props.put(ProducerConfig.ACKS_CONFIG, "1"); // Leader only  
  
// No compression  
props.put(ProducerConfig.COMPRESSION_TYPE_CONFIG, "none");
```

Consumer:

```
Properties props = new Properties();  
  
// Lower poll interval  
props.put(ConsumerConfig.FETCH_MIN_BYTES_CONFIG, "1");  
props.put(ConsumerConfig.FETCH_MAX_WAIT_MS_CONFIG, "10");
```



```
// More frequent polls
// In code: consumer.poll(Duration.ofMillis(10))
```

Broker:

```
# Lower flush intervals (more disk I/O)
log.flush.interval.messages=1
log.flush.interval.ms=10

# Faster leader election
zookeeper.session.timeout.ms=6000
replica.lag.time.max.ms=10000
```

Flink Low-Latency

```
StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

// Lower buffer timeout
env.setBufferTimeout(0); // 0 = no buffering

// Checkpointing (less frequent or disabled for lowest latency)
env.enableCheckpointing(60000, CheckpointingMode.AT_LEAST_ONCE);

// Event time vs processing time
// Processing time = lower latency
DataStream<Event> events = env
    .fromSource(source, WatermarkStrategy.noWatermarks(), "source")
    .keyBy(...)
    .process(new ProcessFunction<Event, Result>() {
        @Override
        public void processElement(Event event, Context ctx, Collector<Result>
out) {
            // Process immediately (no windowing)
            Result result = process(event);
            out.collect(result);
        }
    });
```

Latency Monitoring

```
from prometheus_client import Histogram
import time

# Define latency histogram
processing_latency = Histogram(
```

```

    'event_processing_latency_seconds',
    'End-to-end processing latency',
    ['pipeline'],
    buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)

def process_event(event):
    """Process event and track latency"""
    # Event timestamp (from producer)
    event_timestamp = event['timestamp']

    # Current time
    current_timestamp = time.time()

    # Calculate end-to-end latency
    latency = current_timestamp - event_timestamp

    # Record latency
    processing_latency.labels(pipeline='user-events').observe(latency)

    # Process event
    result = do_processing(event)

    return result

```

6. Capacity Planning {#capacity-planning}

Kafka Capacity Planning

Formula:

Required Partitions = Target Throughput / Partition Throughput

Example:

- Target: 500 MB/s
- Partition throughput: 10 MB/s (typical)
- Required partitions: $500 / 10 = 50$ partitions

Required Brokers = (Partitions * Replication Factor) / Partitions per Broker

Example:

- 50 partitions
- Replication factor: 3
- Partitions per broker: 50 (recommended max)
- Required brokers: $(50 * 3) / 50 = 3$ brokers minimum

Storage Calculation:

$$\text{Storage} = \text{Message Rate} * \text{Message Size} * \text{Retention Period} * \text{Replication Factor}$$

Example:

- Message rate: 10K msg/s
- Average message size: 1 KB
- Retention: 7 days
- Replication factor: 3

$$\begin{aligned} \text{Storage} &= 10,000 * 1\text{KB} * (7 * 24 * 3600) * 3 \\ &= 10,000 * 1\text{KB} * 604,800 * 3 \\ &= 18.1 \text{ TB} \end{aligned}$$

Flink Capacity Planning

Parallelism Calculation:

$$\text{Parallelism} = \text{ceil}(\text{Target Throughput} / \text{Per-Task Throughput})$$

Example:

- Target: 100K events/s
- Per-task throughput: 2K events/s
- Required parallelism: $100\text{K} / 2\text{K} = 50$

$$\text{TaskManagers} = \text{ceil}(\text{Parallelism} / \text{Slots per TaskManager})$$

Example:

- Parallelism: 50
- Slots per TaskManager: 8
- Required TaskManagers: $50 / 8 = 7 \text{ TaskManagers}$

Memory Calculation:

$$\text{TaskManager Memory} = \text{JVM Heap} + \text{Network Buffers} + \text{State} + \text{Overhead}$$

Example:

- JVM Heap: 8 GB
- Network buffers: 2 GB
- State (RocksDB): 4 GB
- Overhead: 2 GB

Total: 16 GB per TaskManager

For 7 TaskManagers: $7 * 16 \text{ GB} = 112 \text{ GB total memory}$

Capacity Planning Tool

File: `capacity_planner.py`

```

class CapacityPlanner:
    """
    Data pipeline capacity planning calculator
    """

    def __init__(self):
        pass

    def calculate_kafka_capacity(
        self,
        message_rate: int,          # messages per second
        message_size: int,          # bytes
        retention_days: int,
        replication_factor: int = 3,
        partition_throughput_mb: float = 10.0
    ) -> dict:
        """Calculate Kafka cluster capacity requirements"""

        # Throughput
        throughput_mb_per_sec = (message_rate * message_size) / (1024 * 1024)

        # Partitions
        required_partitions = max(
            int(throughput_mb_per_sec / partition_throughput_mb),
            1
        )

        # Brokers (assume 50 partitions per broker)
        partitions_per_broker = 50
        total_partition_replicas = required_partitions * replication_factor
        required_brokers = max(
            int(total_partition_replicas / partitions_per_broker),
            replication_factor # Minimum = replication factor
        )

        # Storage
        retention_seconds = retention_days * 24 * 3600
        storage_bytes = message_rate * message_size * retention_seconds *
replication_factor
        storage_tb = storage_bytes / (1024 ** 4)

        # Network bandwidth
        network_bandwidth_mbps = throughput_mb_per_sec * 8 * (1 +
replication_factor)

        return {
            'throughput_mb_per_sec': round(throughput_mb_per_sec, 2),
            'required_partitions': required_partitions,
            'required_brokers': required_brokers,
            'storage_tb': round(storage_tb, 2),
            'network_bandwidth_mbps': round(network_bandwidth_mbps, 2)
        }

```

```

def calculate_flink_capacity(
    self,
    event_rate: int,          # events per second
    per_task_throughput: int,  # events per second per task
    slots_per_taskmanager: int = 8,
    memory_per_taskmanager_gb: int = 16
) -> dict:
    """Calculate Flink cluster capacity requirements"""

    # Parallelism
    required_parallelism = max(
        int(event_rate / per_task_throughput),
        1
    )

    # TaskManagers
    required_taskmanagers = max(
        int(required_parallelism / slots_per_taskmanager),
        1
    )

    # Memory
    total_memory_gb = required_taskmanagers * memory_per_taskmanager_gb

    # CPU (assume 1 vCPU per slot)
    total_vcpus = required_taskmanagers * slots_per_taskmanager

    return {
        'required_parallelism': required_parallelism,
        'required_taskmanagers': required_taskmanagers,
        'slots_per_taskmanager': slots_per_taskmanager,
        'total_memory_gb': total_memory_gb,
        'total_vcpus': total_vcpus
    }

def calculate_spark_capacity(
    self,
    data_size_gb: float,
    processing_time_hours: float = 1.0,
    executor_memory_gb: int = 8,
    executor_cores: int = 4
) -> dict:
    """Calculate Spark cluster capacity requirements"""

    # Parallelism (1 task per GB, roughly)
    required_tasks = int(data_size_gb)

    # Executors
    tasks_per_executor = executor_cores * 2 # 2 tasks per core
    required_executors = max(
        int(required_tasks / tasks_per_executor),
        1
    )

```

```

# Memory
total_memory_gb = required_executors * executor_memory_gb

# CPU
total_vcpus = required_executors * executor_cores

return {
    'data_size_gb': data_size_gb,
    'required_tasks': required_tasks,
    'required_executors': required_executors,
    'executor_memory_gb': executor_memory_gb,
    'executor_cores': executor_cores,
    'total_memory_gb': total_memory_gb,
    'total_vcpus': total_vcpus,
    'estimated_cost_per_hour_usd': self._estimate_cost(total_vcpus,
total_memory_gb)
}

def _estimate_cost(self, vcpus: int, memory_gb: int) -> float:
    """Estimate cloud cost (rough AWS pricing)"""
    # Assume $0.05 per vCPU-hour and $0.01 per GB-hour
    cpu_cost = vcpus * 0.05
    memory_cost = memory_gb * 0.01
    return round(cpu_cost + memory_cost, 2)

# Example usage
if __name__ == "__main__":
    planner = CapacityPlanner()

    # Kafka capacity
    kafka_capacity = planner.calculate_kafka_capacity(
        message_rate=100000,      # 100K msg/s
        message_size=1024,        # 1 KB
        retention_days=7,
        replication_factor=3
    )

    print("Kafka Capacity:")
    print(f"  Throughput: {kafka_capacity['throughput_mb_per_sec']} MB/s")
    print(f"  Partitions: {kafka_capacity['required_partitions']}")
    print(f"  Brokers: {kafka_capacity['required_brokers']}")
    print(f"  Storage: {kafka_capacity['storage_tb']} TB")
    print(f"  Network: {kafka_capacity['network_bandwidth_mbps']} Mbps")

    # Flink capacity
    flink_capacity = planner.calculate_flink_capacity(
        event_rate=100000,        # 100K events/s
        per_task_throughput=2000, # 2K events/s per task
        slots_per_taskmanager=8,
        memory_per_taskmanager_gb=16
    )

    print("\nFlink Capacity:")

```

```
print(f"  Parallelism: {flink_capacity['required_parallelism']}")
print(f"  TaskManagers: {flink_capacity['required_taskmanagers']}")
print(f"  Total Memory: {flink_capacity['total_memory_gb']} GB")
print(f"  Total vCPUs: {flink_capacity['total_vcpus']}")
```

7. Performance Benchmarks {#benchmarks}

Kafka Benchmarks

Setup:

```
# 3-broker cluster
# AWS: 3x r5.2xlarge (8 vCPU, 64 GB RAM, 1 TB SSD each)
# Network: 10 Gbps
```

Producer Benchmark:

```
kafka-producer-perf-test.sh \
  --topic benchmark \
  --num-records 10000000 \
  --record-size 1024 \
  --throughput -1 \
  --producer-props \
    bootstrap.servers=kafka:9092 \
    acks=all \
    compression.type=lz4 \
    batch.size=32768 \
    linger.ms=10

# Results:
# Throughput: 2.1M records/sec (2.1 GB/sec)
# Average latency: 8ms
# p99 latency: 45ms
```

Consumer Benchmark:

```
kafka-consumer-perf-test.sh \
  --bootstrap-server kafka:9092 \
  --topic benchmark \
  --messages 10000000 \
  --threads 10

# Results:
# Throughput: 3.2M records/sec (3.2 GB/sec)
# Average latency: 3ms
```

Flink Benchmarks

Setup:

```
1 JobManager: 4 vCPU, 16 GB RAM
10 TaskManagers: 8 vCPU, 32 GB RAM each
```

Stream Processing Benchmark:

```
// Workload: Count events per user in 5-minute windows
// Input: 1M events/sec, 1KB each

StreamExecutionEnvironment env = ...;
env.setParallelism(80); // 10 TMs * 8 slots

DataStream<Event> events = env.fromSource(...);

DataStream<WindowedCount> counts = events
    .keyBy(Event::getUserId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .aggregate(new CountAggregator());

// Results:
// Throughput: 1.2M events/sec sustained
// End-to-end latency (p50): 120ms
// End-to-end latency (p99): 450ms
// Checkpoint duration: 15 seconds
// State size: 50 GB (RocksDB)
```

Spark Benchmarks

Setup:

```
1 Driver: 4 vCPU, 16 GB RAM
20 Executors: 8 vCPU, 32 GB RAM each
```

Batch Processing Benchmark:

```
# Workload: Process 1 TB of Parquet data
# Operations: Filter, join, aggregate

spark = SparkSession.builder \
    .config("spark.sql.shuffle.partitions", "400") \
    .config("spark.default.parallelism", "400") \
    .getOrCreate()
```



```
df = spark.read.parquet("s3://data/events/") # 1 TB
filtered = df.filter(col("amount") > 100)
aggregated = filtered.groupBy("user_id").agg(sum("amount"))

# Results:
# Processing time: 8 minutes
# Throughput: 2 GB/sec
# CPU utilization: 85%
# Memory utilization: 70%
```

8. Resource Optimization {#resource-optimization}

Cost Optimization Strategies

1. Right-sizing Instances:

```
Oversized (Wasteful):
- 10x r5.4xlarge (16 vCPU, 128 GB)
- CPU utilization: 30%
- Cost: $10K/month

Right-sized (Optimal):
- 15x r5.2xlarge (8 vCPU, 64 GB)
- CPU utilization: 70%
- Cost: $7.5K/month
- Savings: 25%
```

2. Spot/Preemptible Instances:

```
# Kubernetes node pool with spot instances
apiVersion: v1
kind: Node
metadata:
  labels:
    workload: fault-tolerant
    instance-type: spot
spec:
  taints:
    - key: spot
      value: "true"
      effect: NoSchedule
```

3. Auto-scaling:

```
# Scale based on Kafka lag
apiVersion: autoscaling/v2
```

```
kind: HorizontalPodAutoscaler
metadata:
  name: consumer-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: kafka-consumer
  minReplicas: 5
  maxReplicas: 50
  metrics:
    - type: External
      external:
        metric:
          name: kafka_consumer_lag
          selector:
            matchLabels:
              topic: user-events
        target:
          type: AverageValue
          averageValue: "5000"
```

Conclusion

Scaling data ingestion systems requires:

1. **Horizontal scaling** for unlimited growth
2. **Proper partitioning** to distribute load
3. **Backpressure handling** to prevent failures
4. **Throughput optimization** through batching and compression
5. **Latency reduction** when needed
6. **Capacity planning** for cost efficiency
7. **Continuous monitoring** and optimization

Version: 1.0